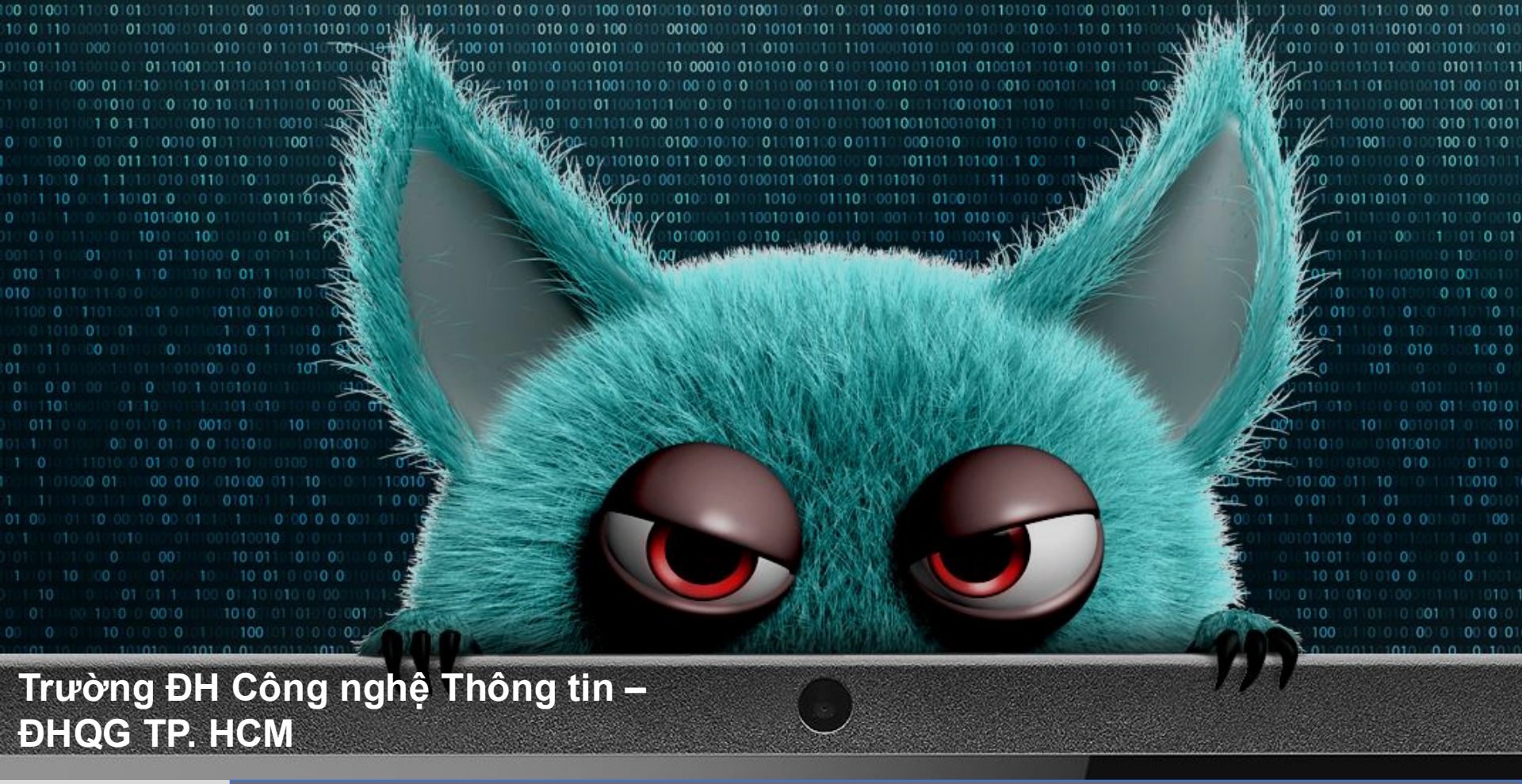
ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



Trường ĐH Công nghệ Thông tin –
ĐHQG TP. HCM

Cơ chế hoạt động của mã độc

NT230 – Malware's Modus Operandi

NT230 - Cơ chế hoạt động của mã độc

Bài giảng lý thuyết – 09.2025

Email: haupv@uit.edu.vn

Computer Virus

Virus máy tính

What are computer viruses?

A computer virus is code that recursively replicates a (possibly evolved) copy of itself.

- Péter Ször, The Art of Computer Virus Research and Defense -

Definition: "A computer virus is a **malicious program** that **attaches itself to a host file** and **replicates itself when the infected file is executed**, often modifying its code to evade detection."

What are computer viruses?

Characteristics:

- **Parasitic**: Requires a "host" (executable file, process, etc.) to attach itself to and propagate.
- **Self-replicating**: Tries to replicate the virus code into other hosts.
- **"Offline" attack vector**: Does not propagate autonomously through a network, but infected files can be transmitted/downloaded by a network.

File injection techniques

- Executable files (*.exe, *.com, *.bat, etc.) are often the target.
- Executing an infected file usually triggers replication of the virus into other.
- **Executable file infection techniques** can be categorized broadly by asking where the virus code is placed in the file.

Basic File Infection Goals

- Enable virus code execution
- Stealth:
  - Preserve original file size
  - Preserve original file functionality – hide from user
  - Pretend to be normal code – hide from anti-virus scanner

File injection techniques

Starting location of the file with destructive overwrite

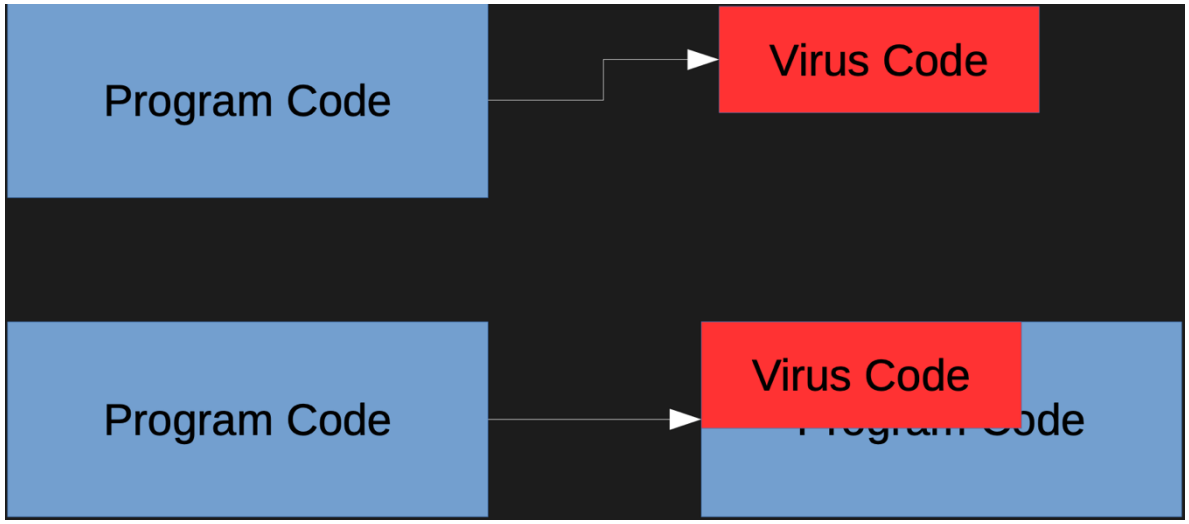
- Beginning always refers to the start of the executable, which might follow a header area in some file formats
- A virus can either preserve the original beginning of the file, or destroy it by overwriting
- Destructiveness always reduces the stealthiness of the virus

File injection techniques

Starting location of the file with destructive overwrite

Two primary methods:

- Replace *.exe file with virus *.exe.
- Overwrite only the beginning of a *.exe that is larger than the virus *.exe.



File injection techniques

Starting location of the file with destructive overwrite

- Neither method is stealthy:
  - The *.exe has lost its functionality entirely, so the user notices that something is wrong
  - Anti-virus software finds a virus easily right at the beginning of the file
- The first method often changes the file size, making it even less stealthy than the second method.

File injection techniques

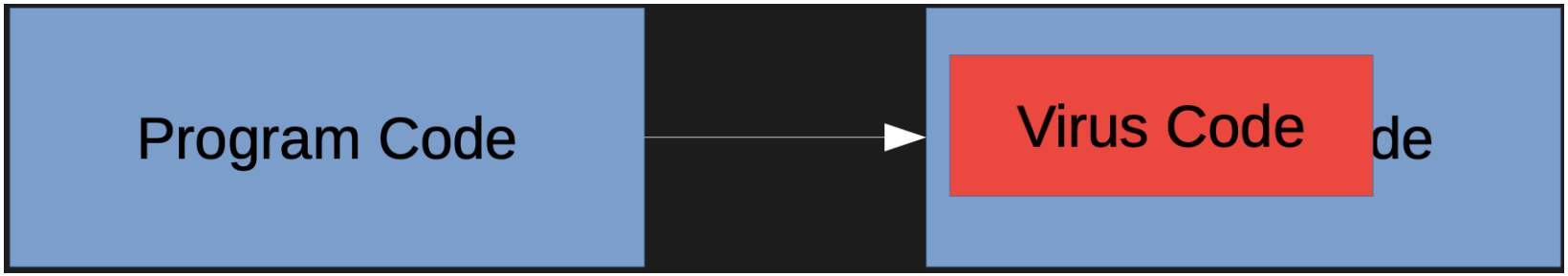
Starting location of the file with destructive overwrite

- The file size change was only significant for stealthy viruses when first-generation anti-virus software depended on keeping track of file sizes
- Both kinds of overwriting viruses can only be repaired by restoring files from a backup
- E.g. the LoveLetter mass mailer worm, after replicating by email, overwrote every file on the system that had one of 32 file extensions: *.c, *.cpp, *.mp3, *.vbs, etc.
  - It had already replicated to other systems, so it no longer tried to remain stealthy; a common design for worms

File injection techniques

Random location of the file with destructive overwrite

- The Russian Omud virus, also called 8888, overwrote at a random location in the * exe file.
  - Anti-virus software must now search the entire file to find it; this defeated early anti-virus software
  - Control might transfer to the virus during execution of the *.exe, or it might not, or program might crash; stealth came at a



File injection techniques

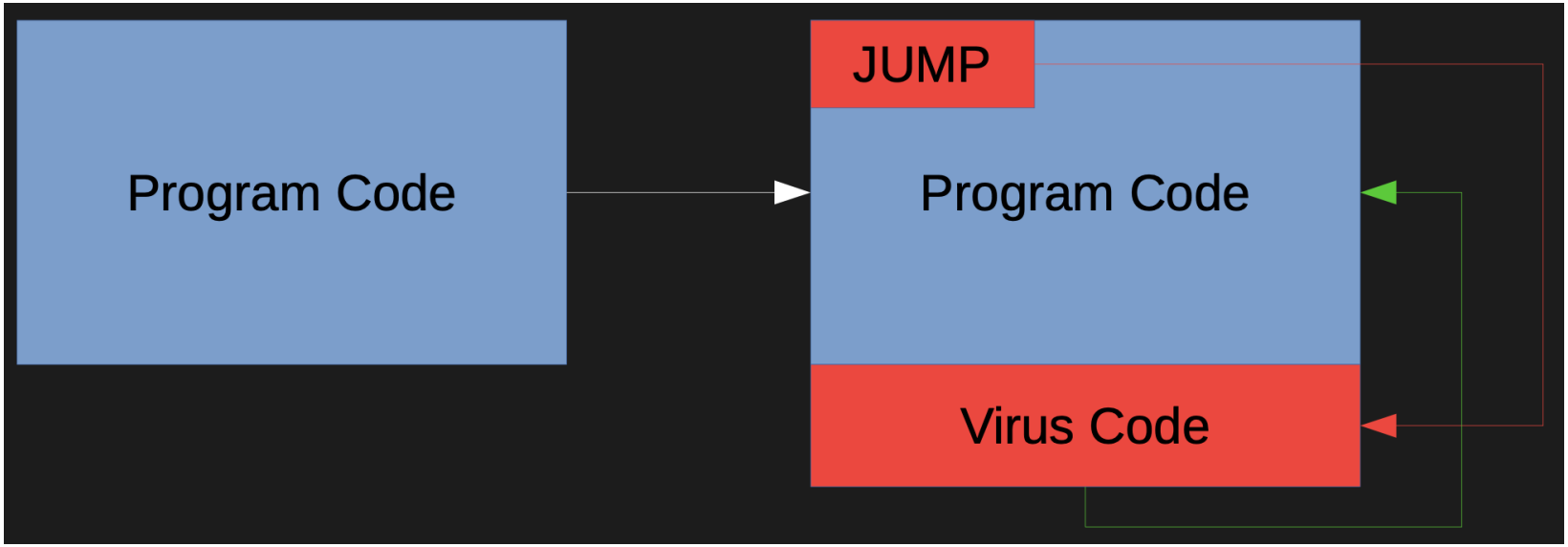
Append to the file

- A jump, or tricky jump, to the virus address is overwritten on the first few bytes of the executable
- Virus code is appended to the file
- Overwritten instructions are saved in the virus
  - When the virus is about to terminate, it executes the saved instructions and jumps back to the spot that followed them
  - Application program functionality is preserved (stealth)
- So common among DOS .COM files that it was called the normal COM virus technique; Vienna and Suicide are famous examples of this kind of virus

File injection techniques

Append to the file

- The jump, or tricky jump, is easily spotted by anti-virus software
- The file size has changed



File injection techniques

Append to the file

- The stealth depends on executing the original application successfully when the virus code has finished, AND not spending too long in the virus code
- In order to execute the application successfully, the virus often copies the application code into a temporary file, then calls system() or a similar function to execute the contents of that file
  - Must pass original command-line arguments!

File injection techniques

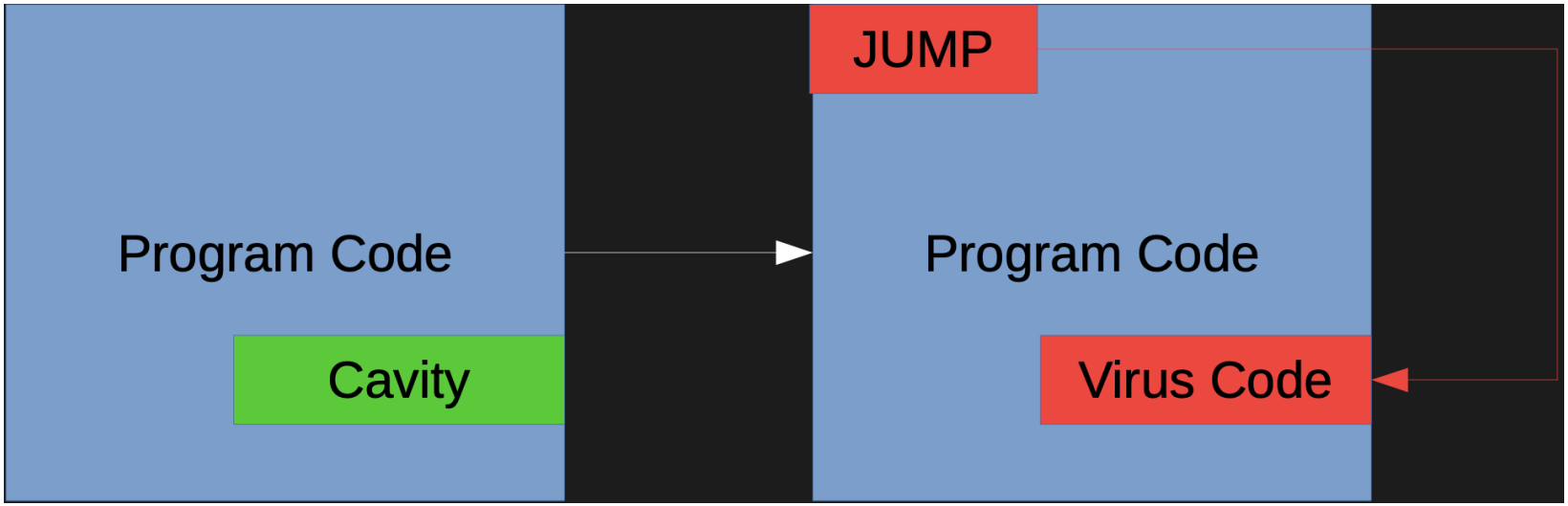
Empty location of the file (cavity)

- Virus creators often search for space within a file that is filled with zeroes or ASCII blanks
- These spaces, or cavities, can be filled with virus code without changing the file size
- A single cavity might be big enough for the whole virus, or the virus might be distributed into multiple small cavities, loaded into memory by the virus loader code at the head of the virus, connected by jump instructions (a fractionated cavity virus)

File injection techniques

Empty location of the file (cavity)

- Jump, or modified PE entry point, detectable by anti-virus software
- Disinfection can be difficult (was the original cavity full of zeroes, or ASCII



File injection techniques

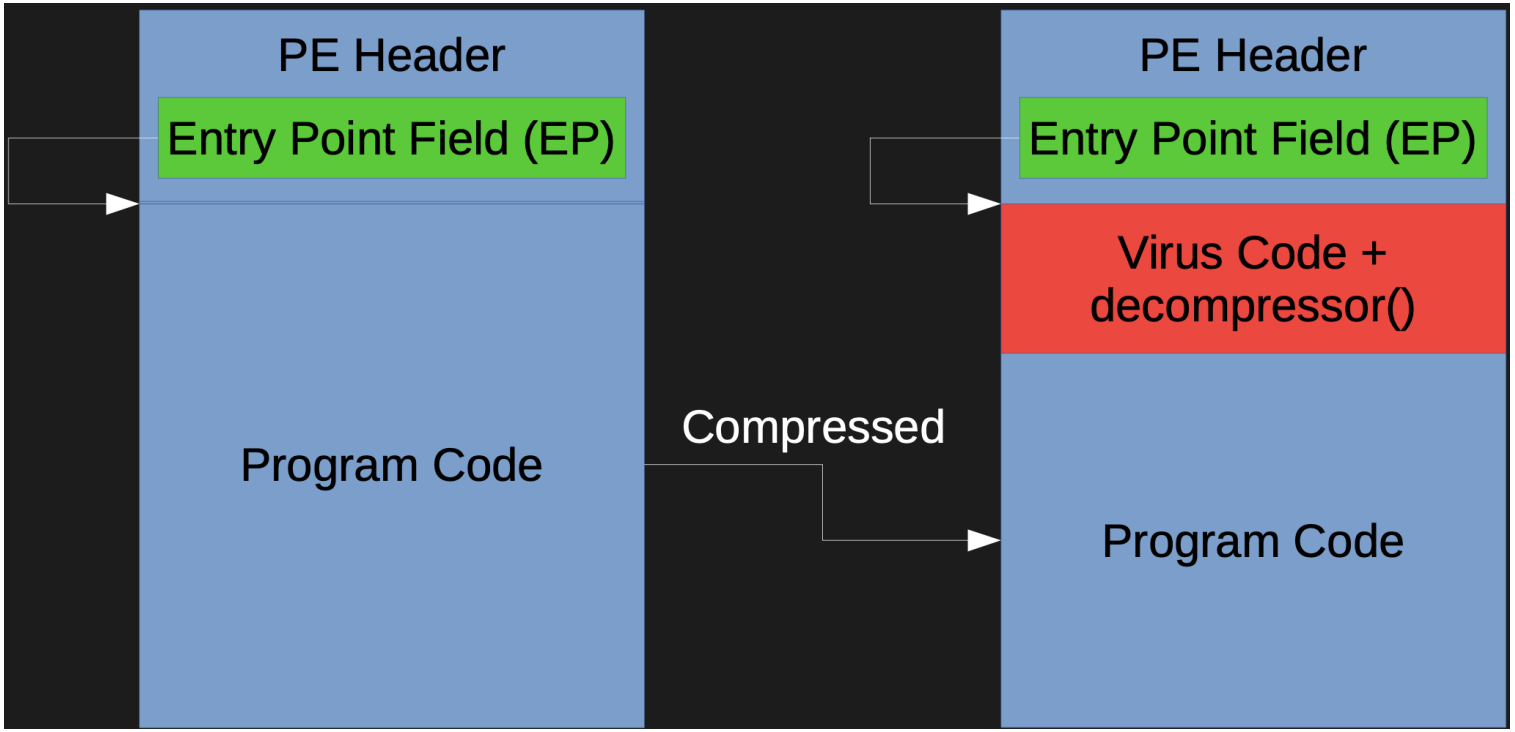
Compressing

- Application code is compressed
- Virus code plus decompressor code fits into the space that was saved
- Can keep the file size from changing
- Might not even change the entry point!

File injection techniques

Compressing

- A self-extracting archive would have a similar appearance and be quite legitimate



File injection techniques

Compressing

- How can a compressing virus be detected and disinfected?
- The virus code might even be compressed, so that only the decompressor code is recognizable as normal code
- File size and entry point could be unchanged
- Application behavior could be preserved

File injection techniques

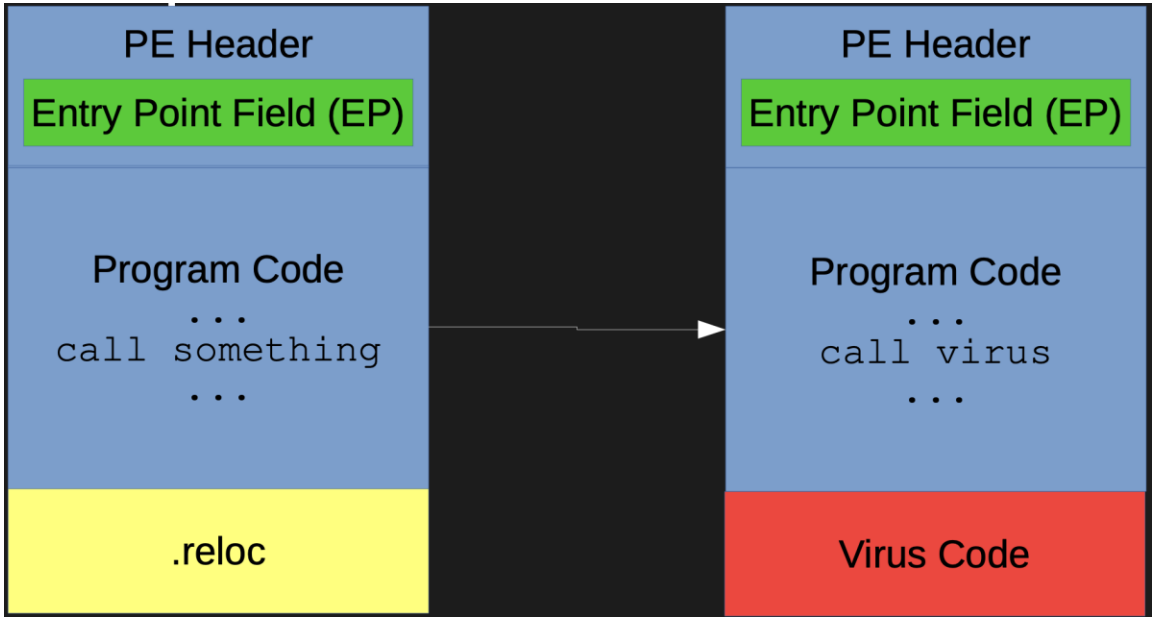
Entry-Point Obscuring (EPO)

- Anti-virus software closely examines PE file headers, entry points, and the initial code executed at the entry point
- A stealthy virus must be designed to avoid changes to any of these locations
- The EPO virus obscures its own entry point by finding a call instruction in the targeted PE file and “hijacking” the call so that the virus code is called instead

File injection techniques

Entry-Point Obscuring (EPO)

- A function call within the application becomes a call to the virus code.



File injection techniques

Entry-Point Obscuring (EPO)

- The virus code saves the registers in order to preserve the parameters that were being passed. Also saves the original call target address.
- When the virus finishes executing, it restores the registers and does a tricky jump back to the original call target.

File injection techniques

Entry-Point Obscuring (EPO)

- How can a virus find a function call?
- The binary opcodes can be scanned. However, constant data in the code section can happen to have the same value as a call opcode
- The most well-designed viruses examine the field that gives the target of the call.
  - What does the virus do with this field?
  - How does this help the virus? (answers in the last slide)

File injection techniques

Entry-Point Obscuring (EPO)

- The .reloc section in PE format gives information to be used if the program has to be relocated during execution, i.e. reloaded at a different load point because the system had to defragment memory or some other reason
- Relocation during execution is unusual, so the .reloc section usually sits unused, e.g. in statically linked executables
- Unfortunately, this provides a large cavity for viruses to use and still leave the file size unchanged

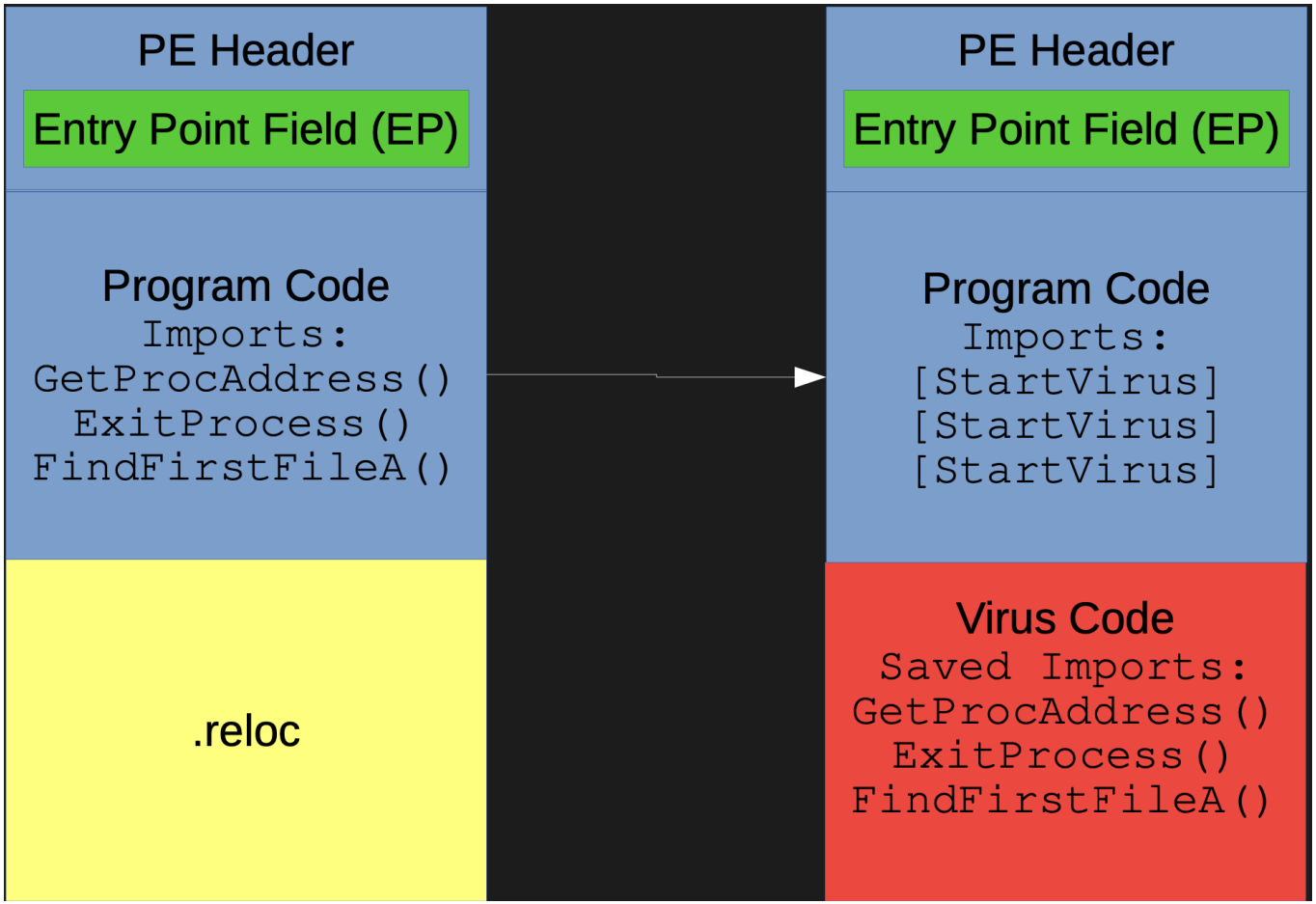
File injection techniques

Import Table-replacing EPO

- The IAT (import address table) is the function pointer table that records the API (application program interface) that the user application is using
- Several function pointers can be saved in the virus body, then replaced with pointers to the virus code
- After the virus code is memory-resident, it can restore the IAT in memory so that the API is preserved and stealth is maintained

File injection techniques

Import Table-replacing EPO



File injection techniques

Semi-EPO: Unknown Entry Points

- Windows PE format has another entry point besides the main entry point – thread local storage (TLS) entry points
- Windows loader search for TLS entry points and execute it first before the main entry point
- TLS entry points can be changed by virus to point to virus code
- Example: W32/Chiton

Advanced Techniques of Virus (Malware)

Một số kỹ thuật nâng cao trong lây nhiễm mã độc

Process injection techniques

- **Process injection** is a method of *executing arbitrary code in the address space of a separate live process*.
- Running code in the context of another process may allow access to the process's memory, system/network resources, and possibly elevated privileges.
- Execution via process injection may also evade detection from security products since the execution is masked under a legitimate process

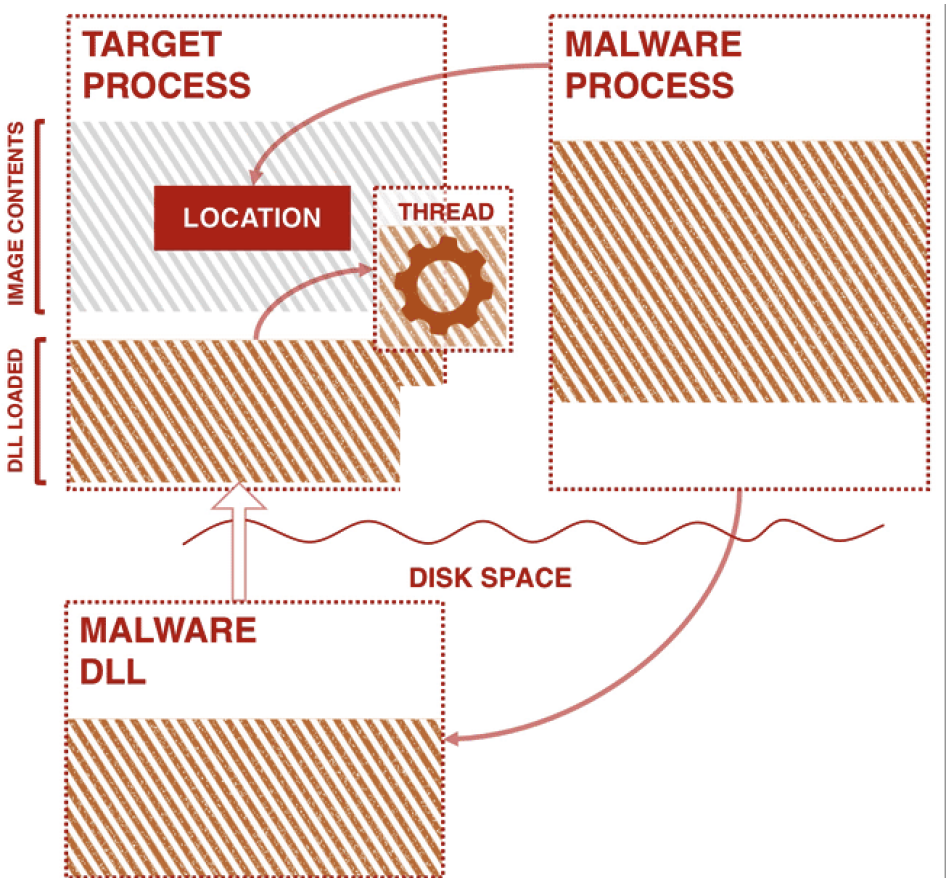
Process injection techniques

- Types:
  - **Process Hollowing**
  - **Process Doppelgänging**
  - Process Herpaderping
  - Thread Execution Hijacking
  - **Reflective DLL Injection**
  - Atom Bombing
  - APC Injection (Asynchronous Procedure Call Injection)
  - Hook Injection (API Hooking để thực thi mã độc)

Process injection techniques

Dynamic-link Library (DLL) Injection

- Is a method of executing arbitrary code in the address space of a separate live process.
- The malware writes the path to its malicious dynamic-link library (DLL) in the virtual address space of another process, and ensures the remote process loads it by creating a remote thread in the target process.



Process injection techniques

Dynamic-link Library (DLL) Injection – How to perform Windows API

- Write: VirtualAllocEx, WriteProcessMemory.
- Invoke: CreateRemoteThread.
  - CreateRemoteThread calls LoadLibrary under the hood to load the DLL

Process injection techniques

Dynamic-link Library (DLL) Injection – Variations

Reflective DLL Injection

- Is a technique that allows an attacker to inject a DLL's into a victim process from **memory** rather than disk.
- Malware will have to manually map the **DLL's raw binary** instead of the path of **on-disk DLL** into virtual memory of target process.

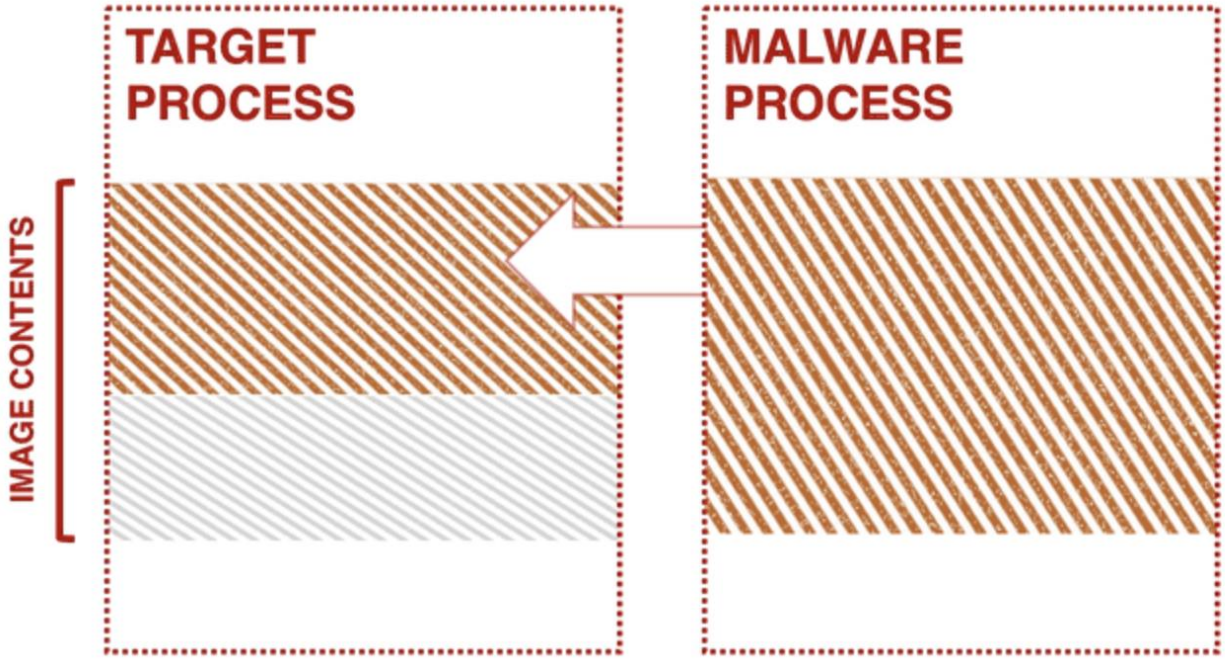
Module Overloading or DLL Hollowing

1. Injects some benign Windows DLL into a remote (target) process
2. Overwrites DLL's, loaded in step 1, AddressOfEntryPoint point with shellcode.
3. Starts a new thread in the target process at the benign DLL's entry point, where the shellcode has been written to, during step 2

Process injection techniques

Process Hollowing

- Performed by creating a process in a suspended state then unmapping/hollowing its memory, which can then be replaced with malicious code.



Process injection techniques

Process Hollowing

Windows API

- Create a suspend process: CreateProcess.
- Unmap the process memory:
  - ZwUnmapViewOfSection,
  - NtUnmapViewOfSection
- Write the malicious code: VirtualAllocEx, WriteProcessMemory.
- Modify Execution Flow: SetThreadContext.
- Resume Execution: ResumeThread.

Process injection techniques

Process Doppelgänging:

- Process Doppelgänging is an **advanced Process Injection technique** that allows malware to replace a legitimate process with malicious code without writing a file to disk.
- **Why is it dangerous?**
  - Leaves no traces on disk (bypasses AV scanning).
  - Does not modify the legitimate process, making detection harder.
  - Malware can disguise itself as legitimate system processes like explorer.exe, svchost.exe.

Process injection techniques

Process Doppelgänging:

- Process Doppelgänging is an **advanced Process Injection technique** that allows malware to replace a legitimate process with malicious code without writing a file to disk.
- **Why is it dangerous?**
  - Leaves no traces on disk (bypasses AV scanning).
  - Does not modify the legitimate process, making detection harder.
  - Malware can disguise itself as legitimate system processes like explorer.exe, svchost.exe.
- **Examples of malware using this technique:**
  - Osiris Banking Trojan
  - SamSam Ransomware
  - Cobalt Strike (hacker attack framework)

Process injection techniques

Steps of Process Doppelgänging:

1. **Create NTFS Transaction (TxF):** Malware creates a hidden file in an NTFS transaction.
2. **Inject malicious code:** Uses NtCreateSection() to load malicious code into memory.
3. **Rollback NTFS Transaction**: The transaction is reverted, the file disappears, but the process remains active.
4. **Execute fake process**: NtCreateProcessEx() executes the process with malicious code.

→The malware runs without a physical file on disk, making it difficult for security tools to detect.

Process injection techniques

Process Doppelgänging:

- Detection Tools:
  - Volatility (analyzing process memory).
  - YARA rules to scan suspicious processes.
  - Moneta (Memory Scanner) to identify abnormal injection behavior.
- Prevention Measures:
  - Disable NTFS Transactions if unnecessary.
  - Monitor suspicious processes utilizing NTFS Transactions.
  - Use forensic tools to detect Memory Injection attempts.

Tài liệu tham khảo

- Wei Wang - CS4630/CS6501 – Defense Against the Dark Arts
- Process injection: https://attack.mitre.org/techniques/T1055/

NT230 – Malware's Modus Operandi

Cơ chế hoạt động của mã độc

Email: inseclab@uit.edu.vn

Trường ĐH Công nghệ Thông tin - ĐHQG TP. HCM

© Trường ĐH Công nghệ Thông tin

Tài liệu giảng dạy môn học